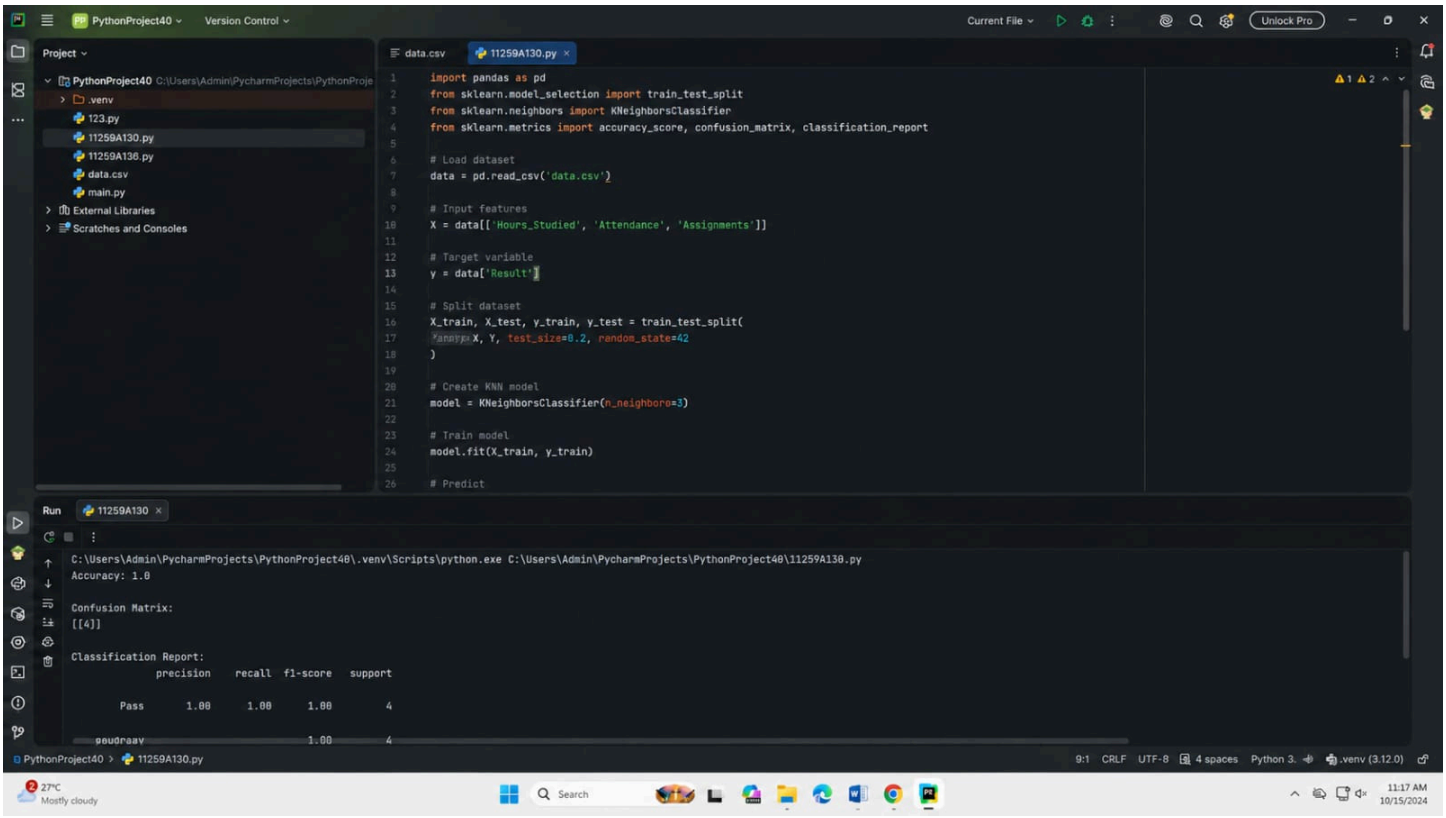
EXNO 4: Implement Classification Model Using KNN and Compute Evaluation Metrics

Aim

To build a K-Nearest Neighbors (KNN) classification model and evaluate its performance using accuracy, confusion matrix, and classification report.

Algorithm

1. Import the required libraries: Pandas, train_test_split, KNeighborsClassifier, and evaluation metrics from Scikit-learn.
2. Load the dataset from the CSV file data.csv.
3. Select the input features Hours_Studied, Attendance, and Assignments and store them in X.
4. Select the target variable Result and store it in y.
5. Split the dataset into 80% training data and 20% testing data using train_test_split() with random_state=42.
6. Create a KNN classifier using KNeighborsClassifier(n_neighbors=3).
7. Train the model using model.fit(X_train, y_train).
8. Predict the target values for the testing data using model.predict(X_test).
9. Calculate and display the model's accuracy using accuracy_score().
10. Display the confusion matrix using confusion_matrix() to compare actual and predicted values.
11. Generate and display the classification report using classification_report(), including precision, recall, F1-score, and support.



Result

The KNN classification model was successfully trained and used to predict the results for the testing dataset. Its performance was evaluated using the following metrics:

- **Accuracy:** Measures the proportion of correct predictions.
- **Confusion Matrix:** Shows the number of correct and incorrect predictions for each class.
- **Classification Report:** Displays precision, recall, F1-score, and support for each class.

Experiment 4: Implement Classification Model Using SVM and Compute Evaluation Metrics

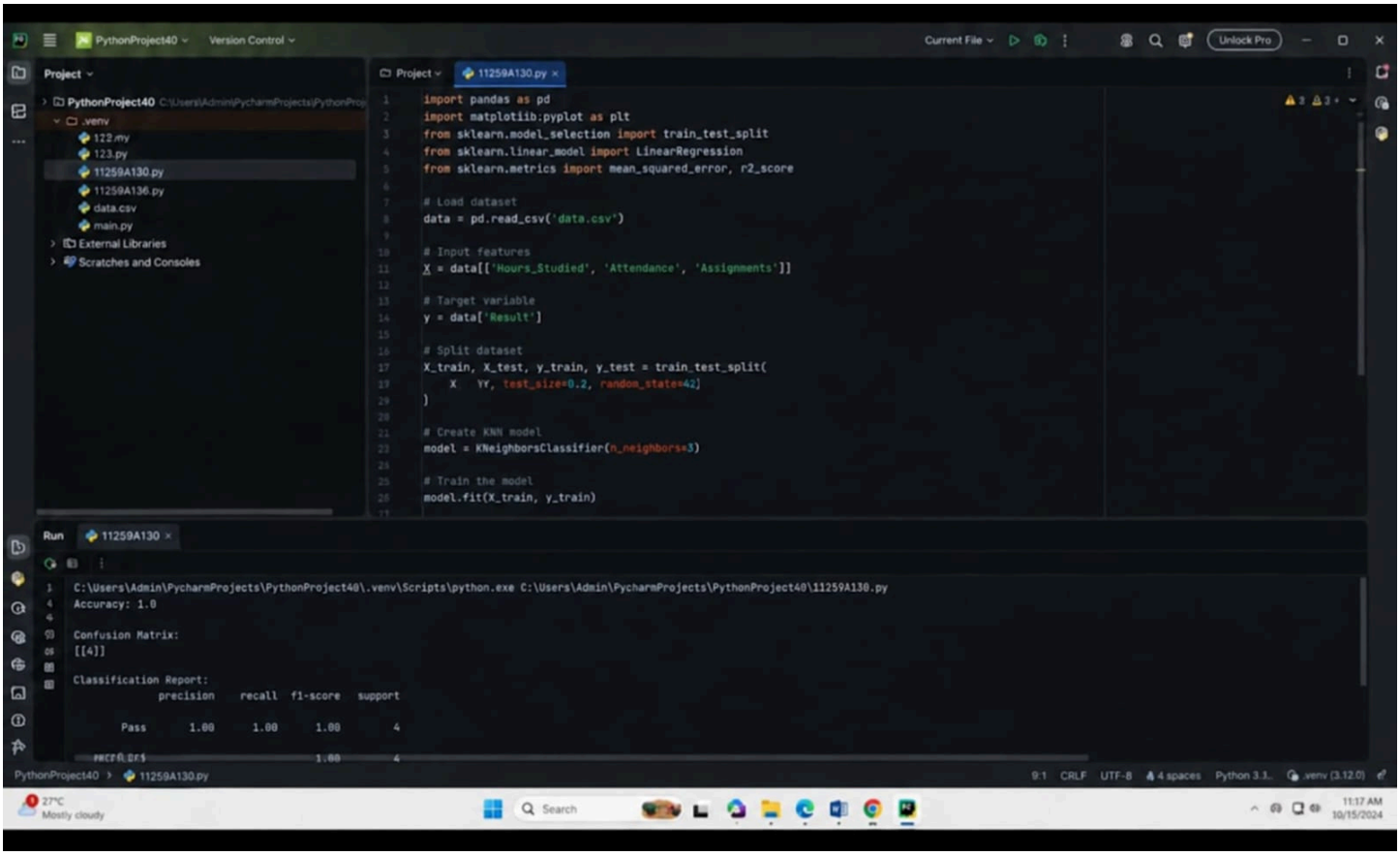
Aim

To implement a Support Vector Machine (SVM) classification model using a linear kernel and evaluate its performance using accuracy, confusion matrix, and classification report.

Algorithm

1. Import the required libraries: Pandas, train_test_split, SVC, and evaluation metrics from Scikit-learn.

2. Load the dataset from the CSV file data.csv.
3. Select the input features Hours_Studied, Attendance, and Assignments and store them in X.
4. Select the target variable Result and store it in y.
5. Split the dataset into 80% training data and 20% testing data using train_test_split() with random_state=42.
6. Create an SVM classification model using SVC(kernel='linear').
7. Train the model using model.fit(X_train, y_train).
8. Predict the target values for the testing data using model.predict(X_test).
9. Calculate and display the prediction accuracy using accuracy_score().
10. Display the confusion matrix using confusion_matrix() to compare actual and predicted classes.
11. Generate and display the classification report using classification_report(), which includes precision, recall, F1-score, and support.



Result

The SVM classification model was successfully trained using a linear kernel and used to predict the results for the testing dataset. The model's performance was evaluated using:

- **Accuracy:** Measures the proportion of correctly classified samples.
- **Confusion Matrix:** Shows the correct and incorrect predictions for each class.
- **Classification Report:** Displays precision, recall, F1-score, and support for each class